

LOKUSQuant: A Language for Low-Latency Quantitative Trading

LOKUSQuant Working Group

Lakshdeep Singh

Ojas Varshney

Aditya Kolgane

Ujwal Gunturi

Mohammad Shozab

Mangamuri Venkata Pranathi

Abstract

LOKUSQuant is a domain-specific language (DSL) designed to developers write ultra low latency code for high frequency trading systems in an easy and intuitive manner. Its allows trading strategies to be written in a much more intuitive and programmable way in terms of financial primitives (like price, spreads, signals, etc.) and higher-level constructs (like regime, cross-asset relationships, etc.) in a declarative manner removing the overhead of manual data feed handling or redundant recalculations reduction. The compiler builds a signal dependency graph (SDG), which is used to eliminate wasted computation and produces optimized code that can adapt to current market regimes. Static checks catch common errors such as type mismatches, invalid position sizing, and inconsistent logic. By using domain specific constraints with compile time optimizations LOKUSQuant makes writing ultra low latency code in a much more productive manner.

Contents

1	Overview	3
2	Problem Statement	3
2.1	Core Inefficiency	3
2.2	Why It Matters	3
2.3	Illustrative Example	3
3	Design Goals	4
4	Language Syntax and Style	4
4.1	Design Philosophy	4
4.2	Data Types	4
4.2.1	Primitive Types	4
4.2.2	Composite Types	5
4.3	Technical Analysis Library	5
4.3.1	Moving Averages	5
4.3.2	Oscillators and Momentum Indicators	5
4.3.3	Volatility and Band Indicators	5
4.4	Mathematical and Statistical Library	6
4.4.1	Aggregation	6
4.4.2	Statistics	6
4.5	Built-in State Variables	6
4.5.1	Position State	6
4.5.2	Portfolio State	6
5	LOKUSQuant: core optimizations and arguments	6
5.1	Signal Dependency Graph (DAG) — formalization	6
5.1.1	Why the DSL level is necessary	7
5.1.2	Example — SimpleMeanReversion	7
5.2	Regime-Aware Pruning	8
5.3	Cross-Asset Arbitrage	9
6	Comparison with other languages and frameworks	9
6.1	Comparison Parameters	9
6.2	Comparison with Pine Script (TradingView)	10
6.3	Comparison with AFL (AmiBroker Formula Language)	10
6.4	Comparison with kdb+/q	10
6.5	Zipline / Backtrader / Python + pandas	11
6.6	QuantLib and numerical libraries	11
6.7	Custom C++/Java engines and proprietary stacks	11
6.8	Where LOKUSQuant sits	12
7	Non-trivial Code Example	12
8	Conclusion	13

1 Overview

LOKUSQuant is designed for the unique demands of quantitative high-frequency trading industry. It has a declarative, strategy-centric syntax that allows programmers to express trading logic directly in terms of financial primitives without having to worry about I/O and the deployment of the strategy. Making such financial primitives first-class data types, LOKUSQuant eliminates boilerplate code and reduces the risk of implementation errors.

Unlike other DSLs in this domain which focus on other aspects, LOKUSQuant's speciality is the integration of compiler-driven optimizations that are domain-aware. The most important optimization of all is the **Signal Dependency Graph** (SDG), which automatically eliminates redundant computations across signals and strategies. Apart from this it has also has other optimizations like regime aware pruning, where only the strategies relevant to current market regime are run, and cross-asset arbitrage detection, enabling the direct expression of multi-asset opportunities.

LOKUSQuant focuses on two main aspects programmability and performance. It provides an intuitive and natural way to write ultra fast low latency code for implementing trading strategies. In doing so, LOKUSQuant provides a unified language for both prototyping and deployment, bridging a gap that has long existed in quantitative finance toolchains.

2 Problem Statement

2.1 Core Inefficiency

In high frequency and low latency quantitative trading systems, microseconds can mean an eternity. The time wasted in useless compute can translate to heavy losses in the real world. But still most of the existing platforms and DSLs provide no easy way to reduce these redundant computations. They don't check whether the underlying inputs have changed or not. This leads to significant redundant computation, especially in strategies with overlapping indicators or cross-asset relationships.

2.2 Why It Matters

At HFT scales, a single strategy may involve hundreds of signals, and desks often deploy dozens of such strategies across thousands of symbols. If overlapping sub-expressions are recalculated each time, the aggregate cost grows quickly. Eliminating this redundancy can yield substantial efficiency gains — not just faster execution, but also higher strategy density per machine.

2.3 Illustrative Example

Consider a trading system where 10 independent strategies are deployed, each defining 20 indicators. If roughly 40% of these indicators overlap across strategies (e.g., multiple SMA windows on the same price series), a naïve engine evaluates them separately for each strategy, leading to hundreds of redundant calculations per tick. At millisecond or microsecond timeframes, this overhead is not merely inconvenient—it becomes a binding constraint on throughput and latency.

3 Design Goals

LOKUSQuant is built on three core principles:

1. **Expressiveness:** declarative syntax for writing strategies with clear modularity using domain native types and a library of optimized primitives for intuitive logic building.
2. **Performance:** SDG based elimination of redundant computation through incremental updates.
3. **Safety:** a domain-aware type system that prevents common trading errors like mixing price/quantity semantics or invalid order parameters. (mixing price and quantity semantics, invalid order fields).

4 Language Syntax and Style

4.1 Design Philosophy

LOKUSQuant adopts a **declarative, domain-centric syntax** that prioritizes readability and financial intuition over computational complexity. The language follows these core design principles:

- **Financial Natural Language Mapping:** Syntax mirrors how quantitative analysts naturally describe trading strategies.

- Buy when price crosses above 20-day average and RSI is below 70:

```
entry_long when {  
    close crosses_above SMA(close, 20) AND rsi < 70  
}
```

- **Hierarchical Strategy Organization:** Strategies are structured as self-contained blocks with clear separation of concerns

```
strategy "momentum_strategy" {  
    parameters  
    indicators  
    entry_long  
    exit_long  
}
```

4.2 Data Types

4.2.1 Primitive Types

```
// Integer  
period: Integer = 20  
  
// Float  
price: Float = 150.75  
  
// Duration  
holding: Duration = 30.days  
exit: Duration = 2.hours
```

```
// Boolean  
flag: Boolean = true
```

4.2.2 Composite Types

```
// TimeSeries  
price_series: TimeSeries[Decimal] = close  
current = close[0]; past10 = close[0:10]  
max_recent = MAX(close, 10)
```

4.3 Technical Analysis Library

4.3.1 Moving Averages

```
// Simple Moving Average  
SMA(series: TimeSeries[Numeric], period: Integer) -> TimeSeries[Float]  
sma_20 = SMA(close, 20)  
  
// Exponential Moving Average  
EMA(series: TimeSeries[Numeric], period: Integer, alpha: Float) ->  
    TimeSeries[Float]  
ema_12 = EMA(close, 12)s
```

4.3.2 Oscillators and Momentum Indicators

```
// Relative Strength Index  
RSI(series: TimeSeries[Numeric], period: Integer) -> TimeSeries[Float]  
rsi = RSI(close, 14) // Returns values 0-100  
  
// Stochastic Oscillator (returns two series)  
StochasticOscillator(high: TimeSeries[Numeric], low: TimeSeries[Numeric],  
    close: TimeSeries[Numeric], k_period: Integer,  
    d_period: Integer)  
    -> (TimeSeries[Float], TimeSeries[Float])  
stoch_k, stoch_d = StochasticOscillator(high, low, close, 14, 3)  
  
// MACD (returns three series)  
MACD(series: TimeSeries[Numeric], fast: Integer, slow: Integer,  
    signal: Integer)  
    -> (TimeSeries[Float], TimeSeries[Float], TimeSeries[Float])  
macd_line, signal_line, histogram = MACD(close, 12, 26, 9)
```

4.3.3 Volatility and Band Indicators

```
// Bollinger Bands (returns three series)  
BollingerBands(series: TimeSeries[Numeric], period: Integer,  
    std_dev: Float)  
    -> (TimeSeries[Float], TimeSeries[Float], TimeSeries[Float])  
bb_upper, bb_middle, bb_lower = BollingerBands(close, 20, 2.0)  
  
// Average True Range  
ATR(high: TimeSeries[Numeric], low: TimeSeries[Numeric],
```

```

    close: TimeSeries[Numeric], period: Integer) -> TimeSeries[Float]
atr = ATR(high, low, close, 14)

// Standard Deviation
STDEV(series: TimeSeries[Numeric], period: Integer) -> TimeSeries[Float]
volatility = STDEV(close, 20)

```

4.4 Mathematical and Statistical Library

4.4.1 Aggregation

```

MAX(series, n) -> TimeSeries[Float]    // max over n
MIN(series, n) -> TimeSeries[Float]    // min over n
SUM(series, n) -> TimeSeries[Float]    // sum over n
MEDIAN(series, n) -> TimeSeries[Float] // median over n
highest = MAX(high, 20)
lowest = MIN(low, 20)

```

4.4.2 Statistics

```

CORRELATION(s1, s2, n) -> TimeSeries[Float]
SLOPE(series, n) -> TimeSeries[Float]
RSQUARED(series, n) -> TimeSeries[Float]
corr = CORRELATION(close, spy_close, 30)
trend = SLOPE(close, 20)

```

4.5 Built-in State Variables

4.5.1 Position State

```

position_shares: Integer           // Number of shares held
entry_price: Float                 // Price at which position was
    entered
days_in_position: Integer         // Days since position entry

```

4.5.2 Portfolio State

```

// Portfolio-level information
portfolio_value: Integer           // Total portfolio value
cash_available: Integer            // Available cash for new positions

```

5 LOKUSQuant: core optimizations and arguments

This section presents the main optimizations introduced and the rationale for why they pay off.

5.1 Signal Dependency Graph (DAG) — formalization

We formalize strategy logic as a DAG $G = (V, E)$ where:

- Every node $v \in V$ is a computation in its smallest reducible format (indicator, rolling statistic, arithmetic expression) or an input leaf (tick, bar, book snapshot).
- Edge from $u \rightarrow v$ means v depends on u .
- Vertices with no outgoing edge are the *output nodes* (signals that trigger orders or logs).

At runtime, we calculate the subset of leaves L that will be affected by a given event. We compute the transitive closure of affected nodes $A(L) = \{v \mid \exists \text{path } \ell \in L \rightarrow v\}$. The runtime evaluates only $A(L)$. If nodes are shared across strategies/assets, they appear in the same global DAG and are evaluated only once instead of multiple times.

5.1.1 Why the DSL level is necessary

This entire optimization technique can be replicated in a any other language but it will require disciplined manual caching and orchestration. The DSL gives three big advantages:

1. Automatic detection of equivalence of expressions (e.g., `SMA(close,20)` written multiple ways).
2. Global optimization across strategies and modules without manual coordination.
3. Safe inference and guards: attach metadata (stateful vs stateless, window sizes, warm-up costs) used in scheduling decisions.

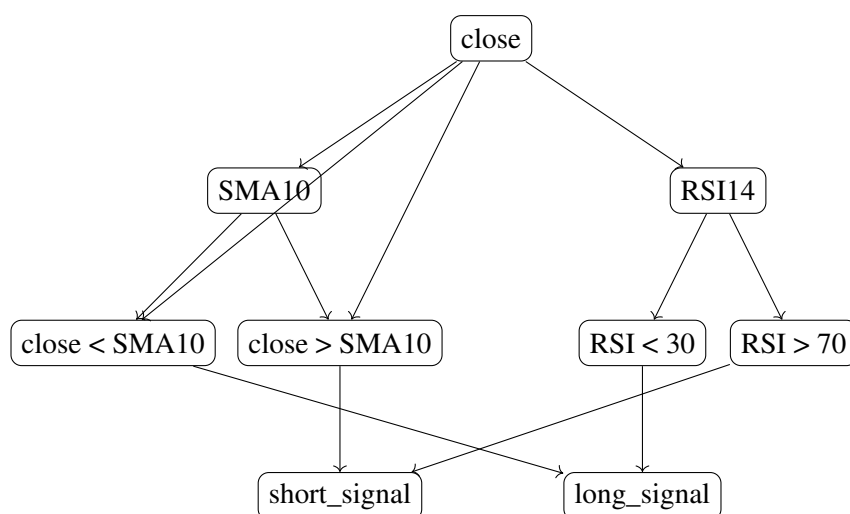
5.1.2 Example — SimpleMeanReversion

We illustrate how the DAG-centric evaluation reduces redundant computation with a minimal example.

Strategy

```
indicators {
  sma10 = SMA(close, 10)
  rsi   = RSI(close, 14)
}
signals {
  long_signal = (close < sma10) && (rsi < 30)
  short_signal = (close > sma10) && (rsi > 70)
}
```

DAG



Key savings

- **Single evaluation of SMA10 and RSI:** both long and short signals reference them; computed once per tick.
- **Shared close series:** both SMA and RSI use the same data; maintain one buffer for both computations.

5.2 Regime-Aware Pruning

The DSL runs only the code made for the current market regime. Only the computations in the branch of active market regime are computed and the rest are pruned. For example, if the current regime is trending then the system ignores the signals in the ranging regime. This reduces unnecessary computations for regimes currently inactive.

Strategy

```

indicators {
  sma20 = SMA(close, 20)
  atr   = ATR(high, low, close, 14)
  rsi   = RSI(close, 14)
}
regimes {
  trending = atr > atr_avg * 1.5
  ranging  = atr < atr_avg * 0.8
}
signals {
  regime trending: long_signal  = sma20 > close && rsi < 40
  regime trending: short_signal = sma20 < close && rsi > 60
  regime ranging:  long_signal  = rsi < 30
  regime ranging:  short_signal = rsi > 70
}
  
```


Key benefits

- Indicators and signals are only evaluated when relevant to the current regime.
- It abstracts regime gating, preventing manual error-prone logic to enable/disable signals.
- Strategy automatically adapts to changing market conditions without developer intervention.

5.3 Cross-Asset Arbitrage

The DSL uses the DAG built for SDG representing both price feeds, derived relations and signals. Rolling stats like `hedge_ratio` and `spread` are computed only once and shared with everyone. This eliminates redundant computations across multiple assets on the same underlying data.

Strategy

```
indicators {
  price_AAPL = close("AAPL")
  price_MSFT = close("MSFT")
}
relationships {
  hedge_ratio = rolling_beta(price_AAPL, price_MSFT, 30)
  spread      = price_AAPL - hedge_ratio * price_MSFT
  z_score     = (spread - mean(spread, 20)) / std(spread, 20)
}
signals {
  long_spread = z_score < -2.0
  short_spread = z_score > 2.0
}
```

Key benefits

- **Single evaluation of shared computations:** Rolling beta, spread, and z-score are computed once per tick and reused.
- **Automatic hedge calculation:** DSL handles dependent quantities, avoiding manual error-prone calculations.
- **Expressive and safe syntax:** DSL ensures type safety and consistent handling of multi-asset relationships.

6 Comparison with other languages and frameworks

In this section we will compare LOKUSQuant against other existing tools and languages in the quant trading domain. The goal is to describe the niche that LOKUSQuant creates for itself in this competitive domain. We will also focus on areas where LOKUSQuant under performs which will serve as motivation for further developments and modifications.

6.1 Comparison Parameters

We compare the languages along the 3 Ps of language design:

- **Performance (P):** The ability to maximize throughput and minimize latency.
- **Portability(P):** The ability to run across research or backtest or production, and integrate with data and execution stacks.
- **Productivity (P):** The ease of writing code specific to that domain in the language, which is facilitated by language syntax and domain specific first class primitives.

6.2 Comparison with Pine Script (TradingView)

Overview: Indicator-first scripting language heavily integrated with charting and publication. It's main targets are retail traders and rapid prototyping.

Strengths:

- Very simple to write indicators and strategies directly on charts.
- Built-in tools for visualization and sharing.

Limitations relative to LOKUSQuant:

- No SDG optimization → each expression is effectively evaluated per bar/tick as written.
- Not for ultra-low-latency trading → Not designed for live execution or multi-asset or co-located arbitrage.
- Controlled runtime → Running on closed platforms and limited portability to production systems.

6.3 Comparison with AFL (AmiBroker Formula Language)

Overview: Backtesting-focused language with heavily optimized vectorized primitives for historical and strategy tests.

Strengths:

- Fast batch backtesting and scanning supported by vectorized C++-backed primitives.
- Trader friendly and familiar formula style syntax.

Limitations relative to LOKUSQuant:

- Made for vectorisation batch processing which is not suited for per-tick incremental evaluation suited for HFTs.
- Does not provide DSL-level cross-asset semantics and model adjusting based on market regimes.
- Attached to AmiBroker platform, therefore production portability limited.

6.4 Comparison with kdb+/q

Overview: Columnar in-memory time-series database with a compact query language. Extremely fast for large time-series queries and analysis.

Strengths:

- Very fast at aggregations and ad-hoc queries over both historical and live tick data.
- Industry proven in HFTs for real-time analysis.

Limitations relative to LOKUSQuant:

- q is a database/query DSL; it is not designed as a strategy-specification language with signals and execution semantics.
- While fast for queries, kdb+ leaves strategisation, incremental DAG optimization, and order-side semantics to the surrounding stack.
- Cross-asset logic and co-located evaluation require careful engineering. The DSL does not natively support signal graphs with subgraphs.

6.5 Zipline / Backtrader / Python + pandas

Overview: Research and backtest frameworks built on Python ecosystem. It is majorly used for prototyping and educational purposes.

Strengths:

- Rich Python ecosystem, easy experimentation, and integration with ML/data tools.
- Familiar to most quant researchers.

Limitations relative to LOKUSQuant:

- Python-level overhead and reliance on vectorized bulk operations makes them unsuitable for live execution that happens in orders of milliseconds.
- No global DAG/CSE optimization strategies and caching must be done manually.
- Migration from research to production requires reimplementing.

6.6 QuantLib and numerical libraries

Overview: Numerical finance libraries (C++/Python bindings), strong for pricing and risk calculations.

Strengths:

- Highly optimized numerical routines and wide coverage of financial instruments.

Limitations relative to LOKUSQuant:

- Not a strategy DSL. QuantLib is complementary (pricing/risk) but does not offer incremental signal graph evaluation.
- Integration would be as a library for specific computations, not as an organizing runtime for many signals/strategies.

6.7 Custom C++/Java engines and proprietary stacks

Overview: Most production trading systems use custom-built engines tailored to particular strategies.

Strengths:

- Maximum control over latency, memory layout, and threading.
- Strategies can reach the absolute lowest latencies with careful planning.

Limitations relative to LOKUSQuant:

- High development cost: every strategy requires careful reimplementing and testing.
- Hard to iterate: Migration from research to production is expensive and vulnerable to errors.
- LOKUSQuant offers a way to retain high-level, shareable specifications while still incorporating many runtime optimizations.

6.8 Where LOKUSQuant sits

LOKUSQuant aims to be the balance point between productivity and performance. It aims to be expressive and familiar for quants (high productivity), but compiler-driven and optimized for live HFT execution (high performance). It trades off some simplicity (introducing a compilation/runtime model) for consistent semantics across research and production, and for automatic optimizations that reduce infrastructure cost.

7 Non-trivial Code Example

```
// SimpleTrade DSL - Moving Average Crossover with RSI Filter and Regime
Detection

strategy MovingAverageCrossover {
  // Basic configuration
  symbol = "AAPL"
  timeframe = {00,00,05}
  p_size = 200000

  // Technical indicators
  indicators {
    fast_ma    = SMA(close, 10)
    slow_ma    = SMA(close, 20)
    rsi_value  = RSI(close, 14)
    volume_avg = SMA(volume, 15)
    atr14      = ATR(high, low, close, 14)
  }

  // Regime detection
  regimes {
    trending = atr14 > SMA(atr14, 20) * 1.5
    ranging  = atr14 <= SMA(atr14, 20) * 1.5
  }

  // Entry conditions
  regime trending: entry_long {
    fast_ma > slow_ma AND
    rsi_value > 40 AND
    rsi_value < 75 AND
    volume > volume_avg
  }

  regime trending: entry_short {
    fast_ma < slow_ma AND
    rsi_value > 25 AND
    rsi_value < 60 AND
    volume > volume_avg
  }

  regime ranging: entry_long {
    rsi_value < 30 AND
    close > 50.0
  }

  regime ranging: entry_short {
```

```
        rsi_value > 70 AND
        close < 200.0
    }

    // Exit conditions
    exit_long {
        fast_ma < slow_ma OR
        rsi_value > 80 OR
        rsi_value < 25 OR
        profit > 12.0 OR
        loss > 6.0
    }

    exit_short {
        fast_ma > slow_ma OR
        rsi_value < 20 OR
        rsi_value > 75 OR
        profit > 10.0 OR
        loss > 5.0
    }
}
```

8 Conclusion

LOKUSQuant is a project that applies existing compiler optimizations and incremental computation techniques to a domain where these optimizations can make a really powerful impact. By providing the ability to write in a declarative syntax which compiles to an optimized runtime code, it reduces duplicated engineering efforts, lowers production costs and shortens time-to-deployment. It has the capability to become a real game changer in the high frequency quantitative trading industry.